Taller práctico Bloque 7

SmartLibrary

Del modelo de clases orientado a objetos a un diseño por tres capas para una API REST con Spring Boot y MongoDB

Asignatura: Diseño de Software · Cuarto semestre

Caso de uso: Registrar préstamo (POST /api/prestamos)

Integrantes: Andrés Camilo Muñoz Bolaños · Walter René Jojoa

Fecha: 11 de noviembre de 2026

Repositorio: https://github.com/walterjojoa/TALLER-PRACTICO-BLOQUE-7

Contenido

1. Diagrama inicial anotado con observaciones
2. Tabla de responsabilidades
3. Diagrama de clases por capas con relaciones y multiplicidades justificadas
4. Diagrama de secuencia del préstamo
5. Proyecto Spring Boot y evidencias de pruebas
6. Decisiones de diseño (una página)
7. Revisión y defensa (Actividad 7)

Anexo. Actividad de cierre: caso Editorial

1. Diagrama inicial anotado con observaciones

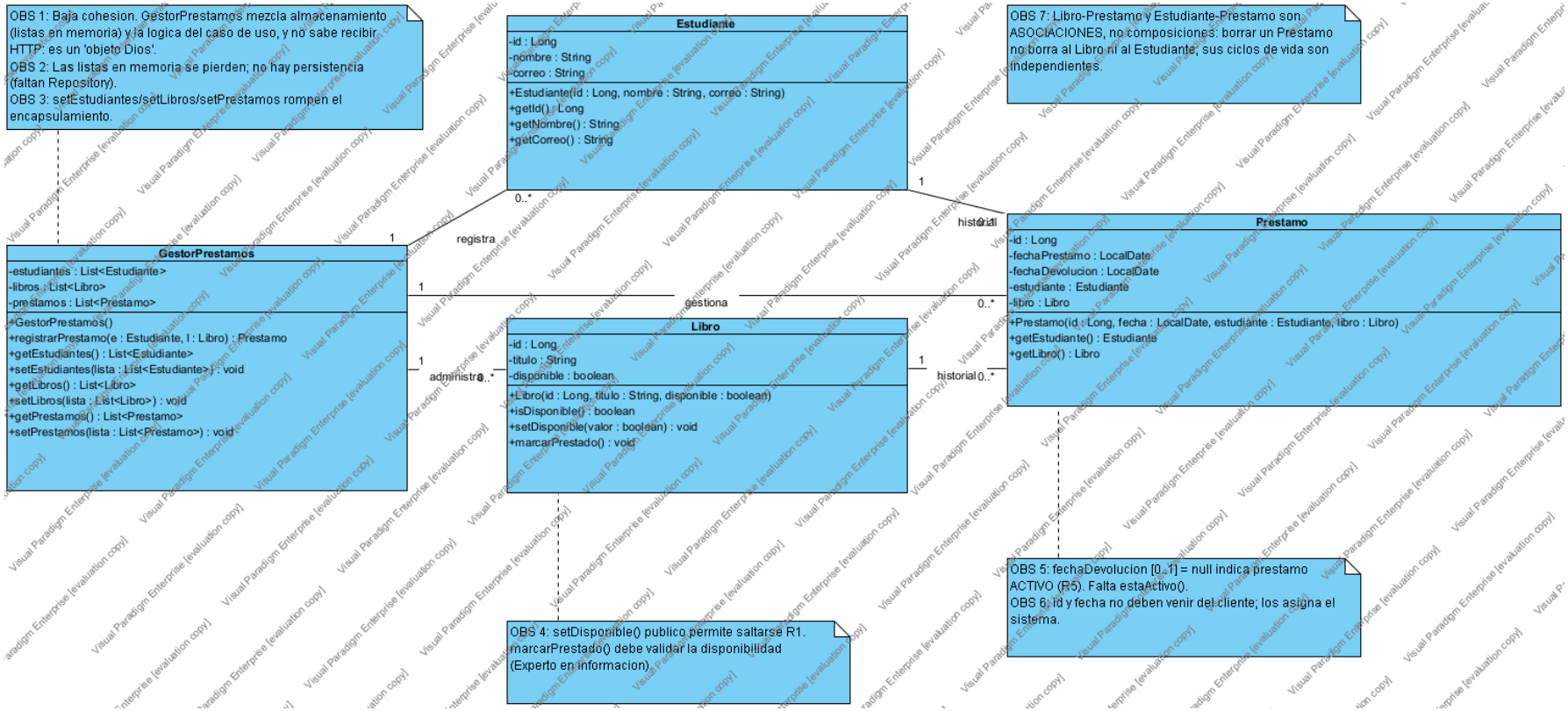


Figura 1. Modelo inicial con las observaciones OBS 1 a OBS 7 (Visual Paradigm).

Observaciones sobre el modelo inicial

1. **OBS 1 · Baja cohesión.** GestorPrestamos mezcla el almacenamiento (listas en memoria) con la lógica del caso de uso y no sabe recibir HTTP: es un «objeto Dios».
2. **OBS 2 · Sin persistencia.** Las listas en memoria se pierden al cerrar la aplicación; faltan repositorios.
3. **OBS 3 · Encapsulamiento roto.** setEstudiantes, setLibros y setPrestamos permiten reemplazar colecciones completas desde afuera.
4. **OBS 4 · Regla expuesta.** setDisponible() público permite saltarse R1. marcarPrestado() debe validar la disponibilidad (Experto en información).
5. **OBS 5 · Préstamo activo.** fechaDevolucion [0..1] en null indica préstamo activo (R5); falta una operación estaActivo().
6. **OBS 6 · Datos del sistema.** El id y la fecha del préstamo no deben venir del cliente; los asigna el sistema.
7. **OBS 7 · Asociaciones, no composiciones.** Borrar un préstamo no borra al libro ni al estudiante: sus ciclos de vida son independientes.

Respuestas de la Actividad 1

- **Datos y operaciones propios del dominio:** Estudiante (id, nombre, correo), Libro (id, título, disponible) y Prestamo (id, fechaPrestamo, fechaDevolucion, estudiante, libro). Operaciones de dominio: Libro.isDisponible(), Libro.marcarPrestado() y la creación de un Prestamo. Las listas de GestorPrestamos no son dominio, son almacenamiento.
- **¿Por qué asociaciones y no composiciones?** En una composición la parte muere con el todo. El libro y el estudiante existen antes del préstamo, siguen existiendo después y participan en muchos préstamos (0..*). Tampoco hay relación todo-parte, así que no es agregación: es una referencia estructural.
- **Operaciones necesarias:** consultar (buscar estudiante y libro por id, listar préstamos, saber si un libro tiene préstamo activo), validar (ids presentes, existencia, disponibilidad) y guardar (préstamo nuevo y libro no disponible).
- **Responsabilidades que faltan:** para HTTP faltan PrestamoController, el DTO PrestamoRequest y un manejador de excepciones que traduzca a 201/404/409; para datos faltan EstudianteRepository, LibroRepository y PrestamoRepository.

2. Tabla de responsabilidades

Distribución de las operaciones del caso de uso entre Controller (entrada/salida HTTP), Service (coordinación) y Repository (persistencia). Las entidades son el modelo del dominio, no una cuarta capa.

Operación o decisión	Clase responsable	Justificación
Recibir POST /api/prestamos	PrestamoController	Frontera HTTP: mapea la ruta, convierte el JSON en PrestamoRequest y lo valida con @Valid. No contiene reglas de negocio.
Buscar estudiante por identificador	EstudianteRepository (lo invoca PrestamoService)	Acceso a datos de una sola colección (findById). El servicio decide qué hacer si no existe (404).
Buscar libro por identificador	LibroRepository (lo invoca PrestamoService)	Igual que el anterior: un repositorio por entidad, alta cohesión.
Validar disponibilidad	PrestamoService + Libro	Regla de negocio (R1, R5): la coordina el servicio; el dato y la verificación pertenecen a Libro.isDisponible() (Experto en información).
Crear el préstamo	PrestamoService	Creador (GRASP): tiene los datos de inicialización (estudiante, libro y fecha actual) (R2).
Cambiar disponibilidad	Libro.marcarPrestado()	Experto en información: el libro conoce y protege su propio estado (R3).
Guardar cambios	PrestamoRepository y LibroRepository	save() en MongoDB después de validar todo (R4): primero el préstamo y luego el libro.
Devolver respuesta HTTP	PrestamoController y ManejadorExcepciones	201 Created con PrestamoResponse; las excepciones se traducen a 404, 409 y los datos inválidos a 400.

3. Diagrama de clases por capas

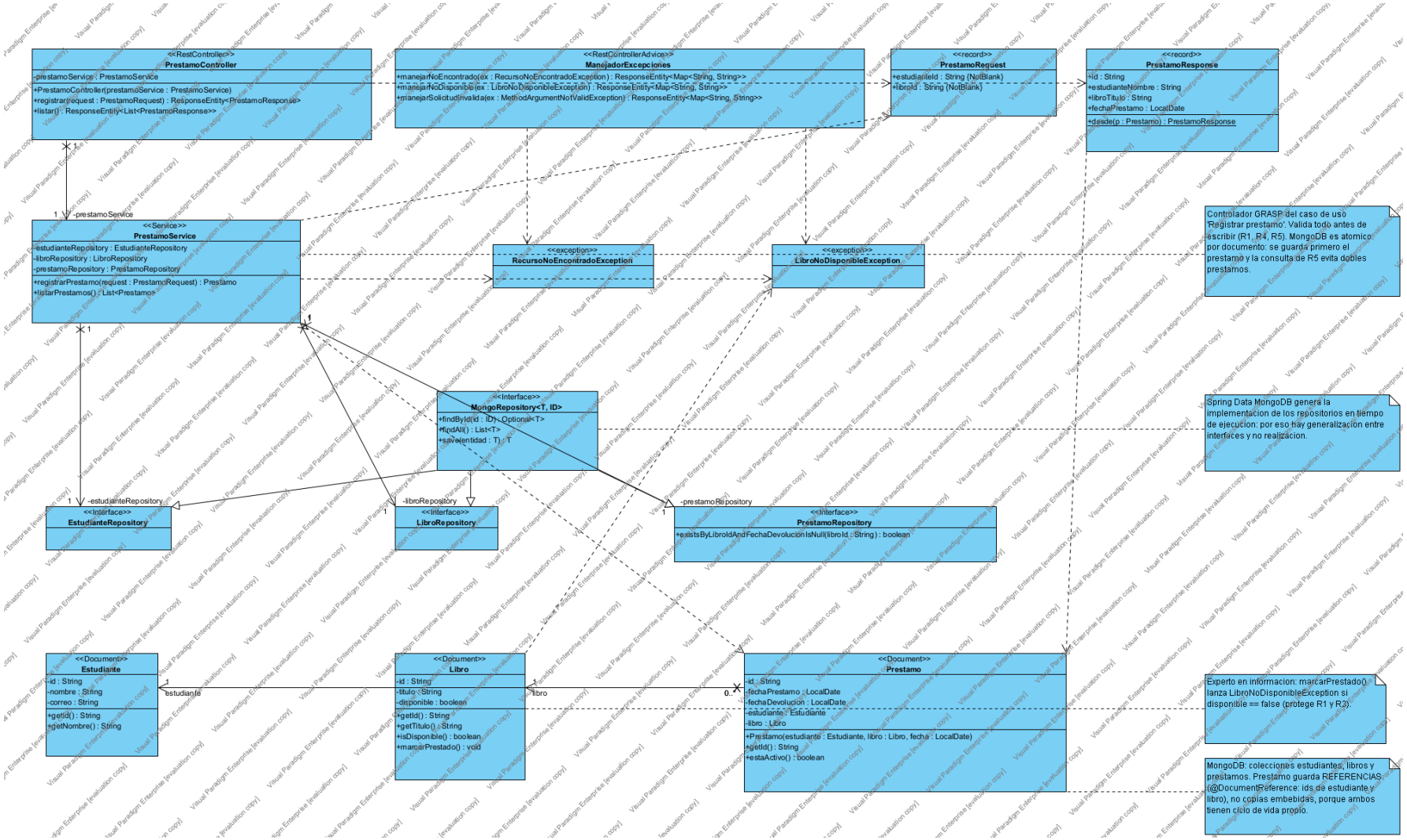


Figura 2. Modelo por capas Controller → Service → Repository con MongoDB (Visual Paradigm).

Justificación de relaciones y multiplicidades

Relación	Tipo UML y multiplicidad	Justificación
PrestamoController → PrestamoService	Asociación dirigida 1 a 1	El controller guarda el servicio como atributo inyectado por constructor (referencia mantenida). La dirección nunca se invierte.
PrestamoService → los 3 repositorios	Asociación dirigida 1 a 1	El servicio mantiene una referencia a cada repositorio. El Controller nunca usa un Repository.
Prestamo → Estudiante y Prestamo → Libro	Asociación 0..* a 1, navegable desde Prestamo	Cada préstamo tiene exactamente un estudiante y un libro; cada uno puede estar en muchos préstamos. En MongoDB se guarda como referencia por id (@DocumentReference).
Repositorios ◁— MongoRepository<T, ID>	Generalización entre interfaces	Los repositorios extienden MongoRepository. No hay realización porque Spring Data genera la implementación en tiempo de ejecución.
Controller ⇢PrestamoRequest / PrestamoResponse	Dependencia «use» / «create»	Uso puntual como parámetro o resultado, sin atributo.
PrestamoService ⇢Prestamo	Dependencia «create»	El servicio crea el préstamo (Creador), pero no lo guarda como atributo.
Service y Libro ⇢excepciones	Dependencia «throw»	Lanzan RecursoNoEncontradoException o LibroNoDisponibleException.
Sin rombos	—	No hay relación todo-parte con ciclo de vida compartido: no se usa agregación ni composición.

Interfaces y clases. Interfaces: EstudianteRepository, LibroRepository, PrestamoRepository y MongoRepository. Clases: el controller, el manejador de excepciones, el servicio, las entidades (@Document), los DTO (records) y las excepciones.

4. Diagrama de secuencia del préstamo

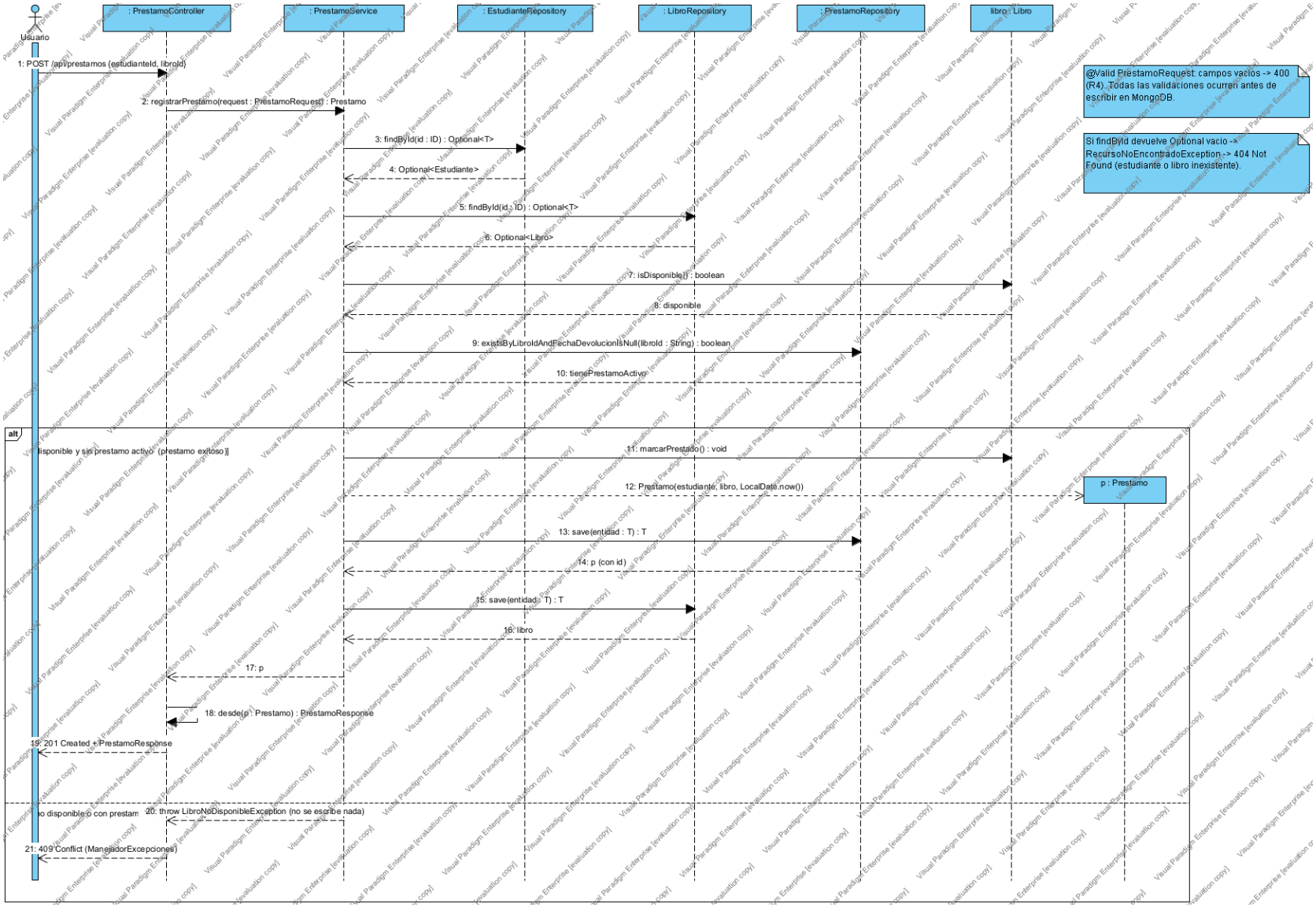


Figura 3. Secuencia de POST /api/prestamos con los caminos «préstamo exitoso» y «libro no disponible» (Visual Paradigm).

Lectura del diagrama de secuencia

Participantes: Usuario, PrestamoController, PrestamoService, EstudianteRepository, LibroRepository, PrestamoRepository, libro:Libro y p:Prestamo. El orden de los mensajes es el mismo de PrestamoService.registrarPrestamo:

1. **Entrada.** El usuario envía POST /api/prestamos; el controller valida el PrestamoRequest (si faltan ids responde 400, R4) y llama a registrarPrestamo(request).
2. **Búsquedas.** El servicio llama a findById en EstudianteRepository y LibroRepository. Si alguno no existe se lanza RecursoNoEncontradoException → 404.
3. **Validación R1 y R5.** isDisponible() sobre el libro y existsByLibroIdAndFechaDevolucionIsNull(libroId) sobre PrestamoRepository.
4. **Camino exitoso.** marcarPrestado(), creación de Prestamo, save(p), save(libro), PrestamoResponse.desde(p) y 201 Created.
5. **Camino «libro no disponible».** Se lanza LibroNoDisponibleException sin escribir nada y el ManejadorExcepciones responde 409 Conflict.

Verificación: cada mensaje corresponde a una operación del diagrama de clases: registrarPrestamo, findById, isDisponible, existsByLibroIdAndFechaDevolucionIsNull, marcarPrestado, el constructor Prestamo(...), save y desde.

5. Proyecto Spring Boot y evidencias de pruebas

Tecnología. Java 21, Maven, Spring Boot 4.1.2, Spring Web, Spring Data MongoDB y Validation. Base de datos: MongoDB 9.0 (servicio local). El enunciado proponía JPA con H2; se usó MongoDB porque así se implementó en la clase y en el repositorio de referencia del profesor. El diseño por capas y las reglas no cambian.

```
smartlibrary/src/main/java/com/smartlibrary/
├── controller/   PrestamoController, ManejadorExcepciones
├── service/      PrestamoService, RecursoNoEncontradoException, LibroNoDisponibleException
├── repository/   EstudianteRepository, LibroRepository, PrestamoRepository  (MongoRepository)
├── model/        Estudiante, Libro, Prestamo  (@Document)
├── dto/          PrestamoRequest, PrestamoResponse
└── config/       DatosIniciales (estudiantes est-1, est-2 y libros lib-1, lib-2, lib-3)
```

Método	Ruta	Cuerpo	Respuestas
POST	/api/prestamos	{"estudianteId": "est-1", "libroId": "lib-1"}	201, 400, 404, 409
GET	/api/prestamos	–	200

Escenarios de la Actividad 6

Escenario	Resultado esperado	Prueba automática	API real + MongoDB
Préstamo de libro disponible	201 Created y préstamo registrado	Aprobada	201
Libro ya prestado	409 Conflict; sin nuevo préstamo	Aprobada	409
Estudiante inexistente	404 Not Found	Aprobada	404
Libro inexistente	404 Not Found	Aprobada	404
Consulta de préstamos	200 OK con registros	Aprobada	200
Extra: solicitud sin libroId (R4)	400 Bad Request	Aprobada	400
Extra: segundo préstamo activo (R5)	409 Conflict	Aprobada	409

Pruebas automáticas. 12 pruebas JUnit 5 aprobadas (0 fallos): PrestamoControllerTest (6, códigos HTTP con MockMvc) y PrestamoServiceTest (6, reglas R1 a R5 con Mockito). Comando: mvnw test.

Pruebas con la API real. Salida de probar-api.cmd contra la API en http://localhost:8080 y MongoDB local (extracto):

```
=== 1. Prestamo de libro disponible (esperado 201 Created)
{"id":"6af4e9d07e9ba30859ab114b","estudianteNombre":"Ana Torres","libroTitulo":"Clean Code","fechaPrestamo":"2026-11-11"}  HTTP
201
=== 2. Libro ya prestado (esperado 409 Conflict)
{"error":"El libro con id lib-1 no esta disponible"}  HTTP 409
=== 3. Estudiante inexistente (esperado 404 Not Found)
{"error":"Estudiante con id est-99 no existe"}  HTTP 404
=== 4. Libro inexistente (esperado 404 Not Found)
{"error":"Libro con id lib-99 no existe"}  HTTP 404
=== 5. Consulta de prestamos (esperado 200 OK con registros)
[{"id":"6af4e9d07e9ba30859ab114b","estudianteNombre":"Ana Torres","libroTitulo":"Clean Code","fechaPrestamo":"2026-11-11"}]  HTTP
200
=== Extra. Solicitud sin libroId (esperado 400 Bad Request, R4)
{"error":"libroId es obligatorio"}  HTTP 400
=== Extra. R5: se fuerza disponible=true en lib-1 y se intenta otro prestamo (esperado 409)
{"error":"El libro con id lib-1 no esta disponible"}  HTTP 409
```

Documento del préstamo guardado en MongoDB (la colección prestamos guarda solo las referencias por id):

```
{ _id: ObjectId('6af4e9d07e9ba30859ab114b'), fechaPrestamo: ISODate('2026-11-11T05:00:00.000Z'),
  estudiante: 'est-1', libro: 'lib-1', _class: 'com.smartlibrary.model.Prestamo' }
```

6. Decisiones de diseño

1. **Tres capas con dependencia en un solo sentido.** Controller → Service → Repository. Cada capa cambia por una sola razón (HTTP, reglas de negocio o persistencia): alta cohesión y bajo acoplamiento. El Controller nunca usa un Repository directamente.
2. **Se elimina GestorPrestamos.** Era un «objeto Dios» con listas en memoria que además coordinaba el caso de uso. Su almacenamiento pasó a los repositorios y su coordinación a PrestamoService; la entrada HTTP, que no existía, quedó en PrestamoController.
3. **PrestamoService es el Controlador GRASP.** Es el primer objeto fuera de la interfaz que recibe la operación del sistema registrarPrestamo y la coordina. El @RestController es presentación: si la petición llegara por otra vía, el controlador GRASP sería el mismo. También es Creador de Prestamo.
4. **La regla de disponibilidad vive en la capa de negocio.** El Service la verifica antes de escribir (R1, R5) y Libro.marcarPrestado() protege el estado del libro (Experto en información). Se eliminó setDisponible() público.
5. **Persistencia en MongoDB con referencias.** Colecciones estudiantes, libros y prestamos. Prestamo guarda referencias (@DocumentReference, solo los ids) y no copias embebidas, porque estudiante y libro tienen ciclo de vida propio y aparecen en muchos préstamos (asociación 0..* a 1).
6. **Consistencia sin transacción entre documentos.** MongoDB es atómico por documento y, sin replica set, no ofrece transacciones entre documentos. Todas las validaciones ocurren antes de escribir (R4); luego se guarda primero el préstamo y después el libro: si lo segundo fallara, la consulta de R5 sigue impidiendo un segundo préstamo. Con MongoDB Atlas se podría añadir MongoTransactionManager.
7. **Repositorios como interfaces.** Extienden MongoRepository; Spring Data genera la implementación. Por eso hay generalización entre interfaces y no realización.
8. **DTO y errores explícitos.** PrestamoRequest trae solo los ids validados con @NotBlank (400 si faltan) y PrestamoResponse evita exponer el documento. Las excepciones de negocio se traducen en un @RestControllerAdvice a 404 y 409, sin condicionales de negocio en el Controller.
9. **Relaciones UML justificadas.** Entre capas, asociaciones dirigidas 1 a 1 (atributos inyectados); en el dominio, Prestamo 0..* → 1 Estudiante y Libro; dependencias «use», «create» y «throw» para usos puntuales sin atributo.
10. **Trazabilidad verificada.** Cada regla tiene su clase, su operación y su prueba: 12 pruebas automáticas y peticiones HTTP reales contra MongoDB.

7. Revisión y defensa (Actividad 7)

- **¿Qué responsabilidades cambiaron?** GestorPrestamos desaparece: el almacenamiento pasó a los repositorios, la coordinación a PrestamoService y la entrada HTTP a PrestamoController. El cambio de estado del libro sigue en Libro.marcarPrestado(), ahora con validación. Se eliminaron los setters de colecciones y el id del constructor de Prestamo.
- **¿Qué patrón GRASP se reconoce?** el patrón Controlador, en PrestamoService. Un controlador GRASP es el primer objeto que recibe y coordina una operación del sistema; el @RestController solo traduce HTTP y pertenece a la presentación. También aparecen Experto en información, Creador, Bajo acoplamiento e Indirección.
- **¿Dónde se ubica la regla de disponibilidad?** en la capa de negocio: PrestamoService la verifica antes de escribir y Libro la protege. No va en el controller (otras entradas podrían saltársela) ni en el repositorio (solo persiste).
- **¿Qué relaciones UML se conservaron y cuáles se modificaron?** se conservan Prestamo–Estudiante y Prestamo–Libro como asociaciones 0..* a 1 sin composición; se vuelven navegables solo desde Prestamo; se eliminan las asociaciones de GestorPrestamos; aparecen las asociaciones 1 a 1 entre capas, la generalización hacia MongoRepository y las dependencias.

¿Cómo se comprueba la trazabilidad requisito → clase → operación → prueba?

Req.	Clase	Operación	Prueba
R1	Libro, PrestamoService	isDisponible(), marcarPrestado()	Libro ya prestado → 409
R2	Prestamo, PrestamoRepository	Prestamo(estudiante, libro, fecha), save()	Préstamo exitoso → 201; consulta → 200
R3	Libro, LibroRepository	marcarPrestado(), save()	Tras el 201 el libro queda no disponible
R4	PrestamoRequest, PrestamoService	@Valid, findById(), validar antes de escribir	400 y 404 sin préstamo creado
R5	Prestamo, PrestamoRepository	estaActivo(), existsByLibroIdAndFechaDevolucionIsNull()	Segundo préstamo activo → 409

Anexo. Actividad de cierre: caso Editorial

Actividad de las diapositivas «De la Web a una API REST con Spring Boot y MongoDB»: Publicacion (abstracta) con los subtipos Libro y Disco, persistidos en una sola colección publicaciones con el discriminador tipo.

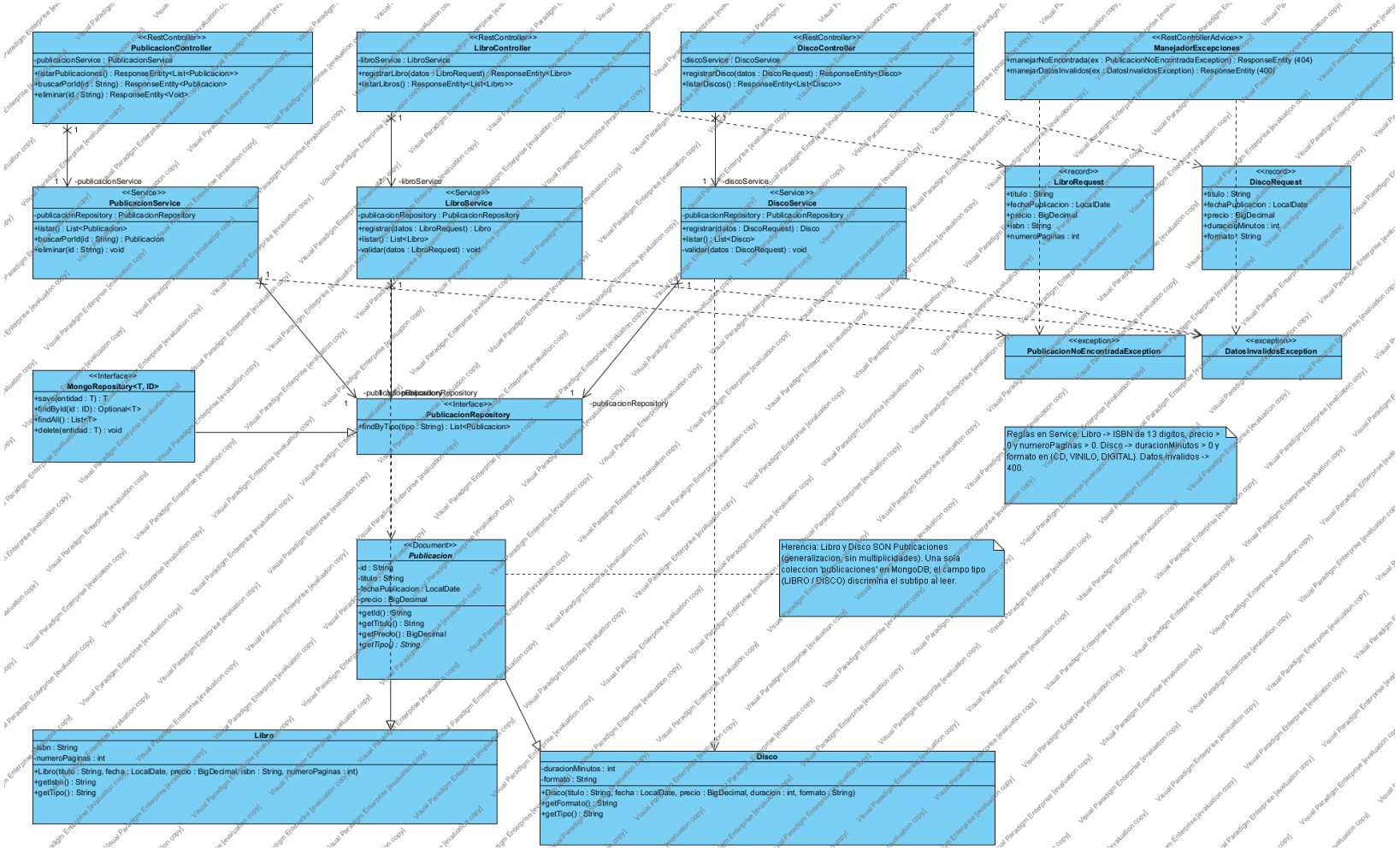


Figura A1. Clases en tres capas con herencia Publicacion → Libro / Disco.

Endpoints propuestos

Método	Ruta	Capacidad	Respuestas
GET	`/api/publicaciones`	Listar publicaciones de ambos tipos	200
GET	`/api/publicaciones/{id}`	Consultar una publicación	200, 404
DELETE	`/api/publicaciones/{id}`	Eliminar una publicación (reto)	204, 404
POST	`/api/libros`	Registrar un libro	201, 400
GET	`/api/libros`	Listar libros	200
POST	`/api/discos`	Registrar un disco	201, 400
GET	`/api/discos`	Listar discos	200

Tabla de responsabilidades

Operación	Controller	Service	Repository
Registrar libro	LibroController.registrarLibro	LibroService.registrar (valida y crea Libro)	PublicacionRepository.save
Registrar disco	DiscoController.registrarDisco	DiscoService.registrar (valida y crea Disco)	PublicacionRepository.save
Listar libros / discos	LibroController / DiscoController	LibroService.listar / DiscoService.listar	findByTipo
Listar publicaciones	PublicacionController.listarPublicaciones	PublicacionService.listar	findAll
Consultar por id	PublicacionController.buscarPorId	PublicacionService.buscarPorId	findById
Eliminar	PublicacionController.eliminar	PublicacionService.eliminar	findById + delete

Justificación: cada capa responde una sola pregunta: cómo entra y sale la información (Controller), qué reglas exige el caso de uso (Service) y cómo se guarda en MongoDB (Repository). Hay un solo repositorio porque ambos tipos comparten la colección publicaciones, y un servicio por tipo porque sus reglas son distintas.

¿Por qué el Controller no guarda directamente en MongoDB? se saltaría las validaciones de negocio, quedaría acoplado a HTTP y a MongoDB a la vez, perdería cohesión y sería difícil de probar. Por la misma razón el navegador nunca se conecta directo a la base de datos.

Reglas validadas en Service. Libro: ISBN de 13 dígitos y numeroPaginas > 0. Disco: duracionMinutos > 0 y formato en {CD, VINILO, DIGITAL}. Ambos: título no vacío y precio > 0. Si no se cumplen, 400 Bad Request.

Reto: flujo de DELETE /api/publicaciones/{id}

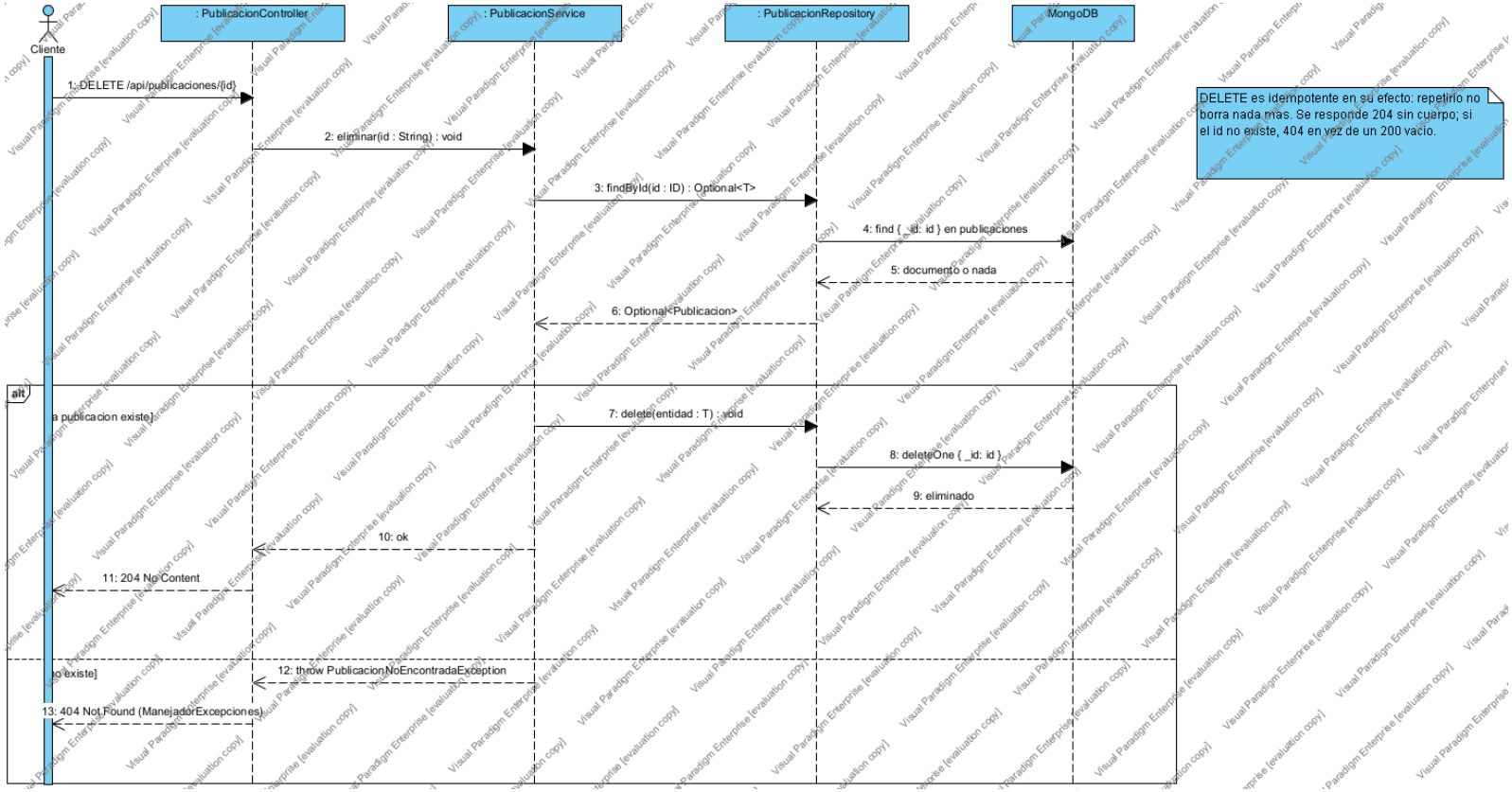


Figura A2. El servicio busca la publicación; si existe la elimina y responde 204, si no existe responde 404.

Secuencia de POST /api/libros

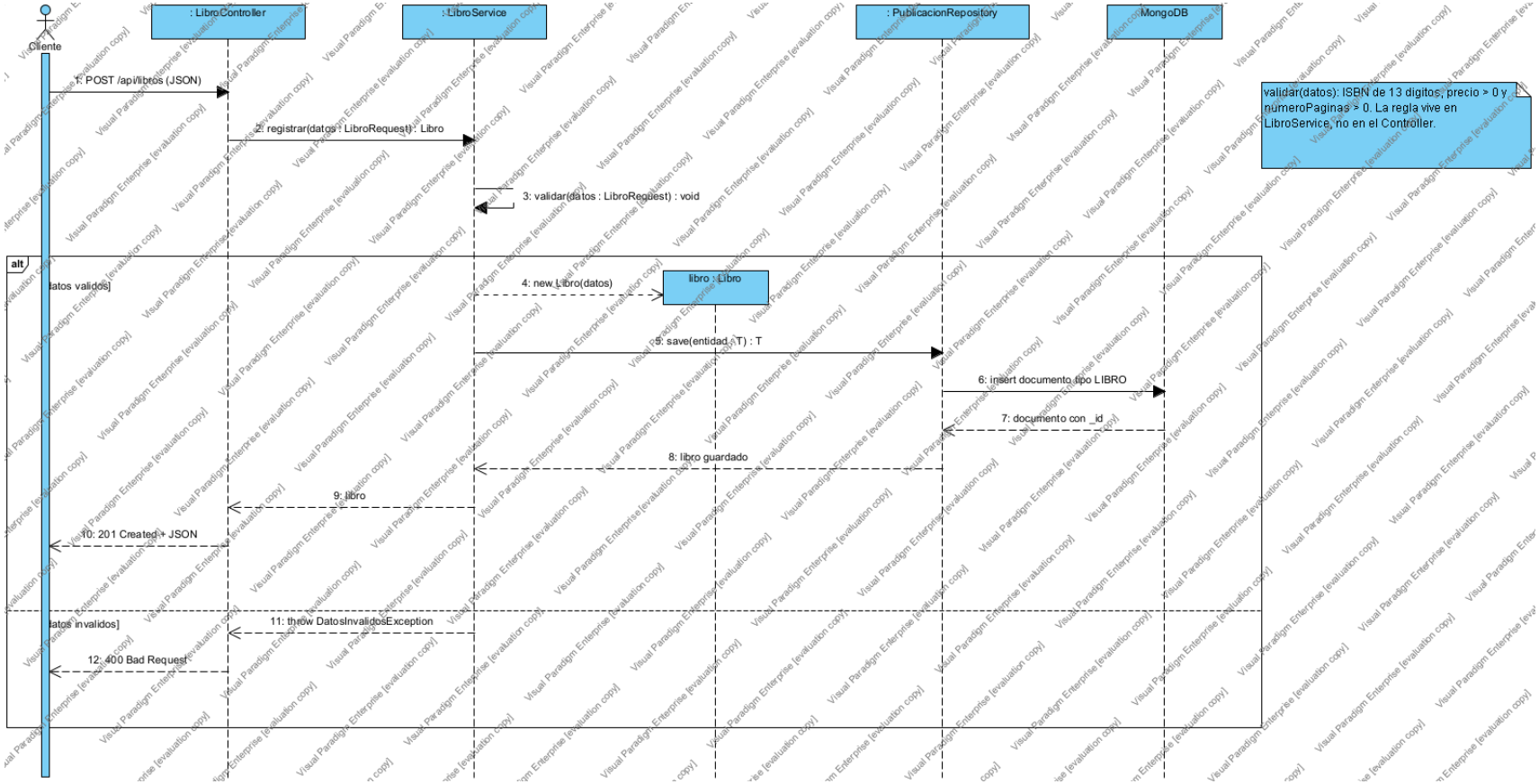


Figura A3. La validación de las reglas del libro ocurre en LibroService: datos válidos → 201, inválidos → 400.